

非同步函數 `async ...await fetch` 範例

`fetchProducts` 函數是一個非同步函數(`async ()`)，它使用 `fetch` API 從後端 API 獲取產品數據。一旦獲取到數據，我們將數據設定為 `products` 的新值。

在渲染部分，我們根據 `products` 的長度來判斷顯示不同的內容。如果 `products` 是空陣列，則顯示一個消息表示目前沒有產品。如果 `products` 不是空陣列，則使用 `map` 方法遍歷每個產品，並以列表的形式顯示產品的名稱和價格。

我們使用 `useState` Hook 來建一個名為 `products` 的狀態變數，並使用空陣列作為初始值。我們使用 `setProducts` 函數來更新 `products` 的值。

在 `useEffect` Hook 中，我們傳遞一個函數和一個依賴陣列 [`products`]。這表示當 `products` 的值發生變化時，這個函數將被執行。在這個例子中，當 `products` 的值改變時，我們呼叫 `fetchProducts` 函數來獲取產品數據。

```
import React, { useEffect, useState } from 'react';
const ProductList = () => {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    fetchProducts();
  }, [products]);

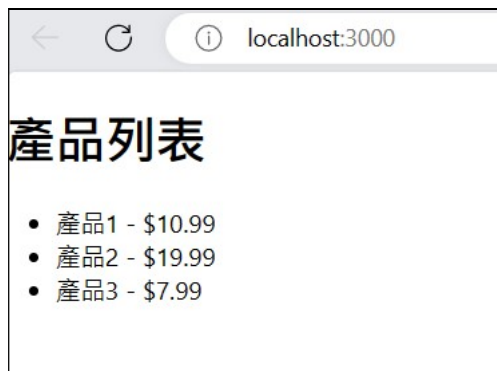
  const fetchProducts = async () => {
    try {
      const response = await fetch('http://localhost:8080/api/products');
      const data = await response.json();
      setProducts(data);
    } catch (error) {
      console.error('Error fetching products:', error);
    }
  };

  return (
    <div>
      <h1>產品列表</h1>
      {products.length === 0 ? (
        <p>目前沒有產品</p>
      ) : (
```

```

        <ul>
            {products.map((product) => (
                <li key={product.id}>
                    {product.name} - ${product.price}
                </li>
            ))}
        </ul>
    ))
</div>
);
};
export default ProductList;

```



後端 spring boot:

```

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.*;
import com.example.demo.model.*;

import jakarta.annotation.PostConstruct;

@RestController

```

```
@RequestMapping("/api/products")
```

```
@CrossOrigin
```

```
public class ProductController {
```

```
    private List<Product> products = new ArrayList<>();
```

```
    @PostConstruct
```

```
    public void init() {
```

```
        // 添加一些範例產品資料
```

```
        products.add(new Product("1", "產品 1", 10.99));
```

```
        products.add(new Product("2", "產品 2", 19.99));
```

```
        products.add(new Product("3", "產品 3", 7.99));
```

```
    }
```

```
    @GetMapping
```

```
    public List<Product> getProducts() {
```

```
        return products;
```

```
    }
```

```
    @GetMapping("/{id}")
```

```
    public ResponseEntity<Product> getProduct(@PathVariable String id) {
```

```
        Product product = findProductById(id);
```

```
        if (product != null) {
```

```
            return ResponseEntity.ok(product);
```

```
        } else {
```

```
            return ResponseEntity.notFound().build();
```

```
        }
```

```
    }
```

```
    @PostMapping
```

```
    public ResponseEntity<Product> createProduct(@RequestBody Product product)
    {
```

```
        products.add(product);
```

```
        return ResponseEntity.ok(product);
```

```
    }
```

```
    @PutMapping("/{id}")
```

```

    public ResponseEntity<Product> updateProduct(@PathVariable String id,
@RequestBody Product updatedProduct) {
        Product product = findProductById(id);
        if (product != null) {
            product.setName(updatedProduct.getName());
            product.setPrice(updatedProduct.getPrice());
            return ResponseEntity.ok(product);
        } else {
            return ResponseEntity.notFound().build();
        }
    }

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteProduct(@PathVariable String id) {
    Product product = findProductById(id);
    if (product != null) {
        products.remove(product);
        return ResponseEntity.ok().build();
    } else {
        return ResponseEntity.notFound().build();
    }
}

private Product findProductById(String id) {
    return products.stream()
        .filter(product -> product.getId().equals(id))
        .findFirst()
        .orElse(null);
}
}

```